

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №2
з дисципліни: «Поглиблене вивчення JavaScript»
з теми: «Розробка галереї зображень із використанням API»

Виконав:
ст. гр. ПЗПІ-23-2
Малишкін Андрій

Перевірів:
посада викладача (уточнити)
ПІБ викладача (уточнити)

2 РОЗРОБКА ГАЛЕРЕЇ ЗОБРАЖЕНЬ ІЗ ВИКОРИСТАННЯМ API

2.1 Мета роботи

Метою роботи є ознайомлення з механізмами асинхронного програмування в JavaScript, опанування способів взаємодії з зовнішніми API засобами Vue 3, формування практичних навичок обробки асинхронних даних, станів завантаження та помилок у сучасних SPA-застосунках.

2.2 Завдання

У межах лабораторної роботи необхідно:

- підключитися до зовнішнього API Picsum Photos та завантажити список зображень;
- відобразити зображення у вигляді сітки карток із підписом автора;
- реалізувати стан завантаження з індикатором;
- реалізувати обробку та відображення помилок запиту з можливістю повторної спроби;
- забезпечити додавання зображень до обраних та їх видалення;
- реалізувати пошук за іменем автора через обчислювану властивість;
- реалізувати фільтрацію між режимами «Усі» та «Обрані».

2.3 Хід роботи

2.3.1 Теоретичні відомості

Було розглянуто асинхронне програмування як підхід до виконання операцій, що потребують часу (зокрема мережових запитів), без блокування інтерфейсу. Для роботи з такими операціями використовуються об'єкти «Promise» та синтаксис «async/await», що дозволяє описувати асинхронний код у лінійному стилі.

Для HTTP-запитів застосовано вбудований метод «fetch» із перевіркою «response.ok» та подальшим парсингом JSON. Окремо було опрацьовано обробку помилок через «try/catch/finally» та організацію стану завантаження («isLoading») для інформування користувача.

У Vue 3 для ініціалізації даних після монтування компонента використано хук «onMounted». Для пошуку та фільтрації застосовано «computed», що автоматично перераховує похідний масив зображень при зміні параметрів.

2.3.2 Етапи реалізації застосунку

Під час практичної реалізації було послідовно виконано такі етапи:

- підготовлено проєктну структуру Vue 3 + TypeScript у директорії «src»;
- створено модуль «api/picsum.ts» для запитів до «https://picsum.photos/v2/list»;
- організовано реактивний стан: «images», «isLoading», «error», «favoriteIds», «searchQuery», «viewMode»;
- реалізовано функцію «loadImages()» з «async/await» та виклик у «onMounted»;
- реалізовано умовне відображення індикатора завантаження, повідомлення про помилку та галереї;
- реалізовано сітку карток із підписом автора та кнопкою обраного;
- реалізовано пошук за автором і перемикання режимів «Усі» / «Обрані» через «computed»;
- оформлено інтерфейс у темній темі з використанням Tailwind CSS та shadcn-подібних UI-компонентів.

2.3.3 Опис архітектури та компонентів

Застосовано компонентну декомпозицію:

- «App.vue»: керування станом, завантаженням даних, фільтрами та пошуком;
- «GalleryGrid.vue»: відображення сітки карток і порожнього стану;
- «ImageCard.vue»: окрема картка зображення з підписом автора та кнопкою обраного;
- «api/picsum.ts»: ізольований шар HTTP-запитів до API;
- «UiButton.vue», «UiInput.vue», «UiCard.vue»: базові UI-примітиви для уніфікації вигляду елементів.

Такий поділ дозволив відокремити мережеву логіку від представлення та спростити підтримку коду.

2.3.4 Короткі фрагменти програмного коду

Нижче наведено ключові фрагменти реалізації, що демонструють основні принципи роботи застосунку.

2.3.4.1 Завантаження зображень з API

```

async function loadImages(): Promise<void> {
  isLoading.value = true
  error.value = null

  try {
    images.value = await fetchImages(1, 20)
  } catch (loadError) {
    error.value =
      loadError instanceof Error
        ? loadError.message
        : 'Не вдалося завантажити зображення'
  } finally {
    isLoading.value = false
  }
}

onMounted(() => {
  void loadImages()
})

```

2.3.4.2 Фільтрація та пошук через обчислюване значення

```

const visibleImages = computed(() => {
  let list = images.value

  if (viewMode.value === 'favorites') {
    list = list.filter((image) =>
      favoriteIds.value.includes(image.id))
  }

  const normalizedQuery = searchQuery.value.trim().toLowerCase()
  if (normalizedQuery) {
    list = list.filter((image) =>
      image.author.toLowerCase().includes(normalizedQuery),
    )
  }

  return list
})

```

2.4 Результати роботи

У результаті виконання лабораторної роботи отримано працездатну галерею зображень на Vue 3. Було забезпечено завантаження даних із Picsum Photos API, відображення сітки карток, індикатор завантаження, обробку помилок із повторною спробою, локальний список обраних, пошук за автором і фільтрацію за режимами «Усі» та «Обрані».

Під час перевірки працездатності було підтверджено:

- коректне завантаження та відображення зображень;
- відображення індикатора завантаження під час запиту;
- коректну роботу обраних і фільтрації;
- відображення помилки та повторне завантаження при невдалому запиті;
- відсутність помилок lint/typescheck у проєкті.

2.5 Висновки

Під час виконання лабораторної роботи було закріплено практичні підходи до асинхронної взаємодії з API у Vue 3. Було відпрацьовано застосування «fetch», «async/await», «onMounted», «computed» та компонентного підходу, а також отримано навички організації станів завантаження і помилок у SPA.

2.6 Контрольні запитання

- а) Що таке асинхронне програмування і чому воно необхідне у вебзастосунках?

Асинхронне програмування — це виконання операцій без блокування основного потоку. У вебзастосунках воно необхідне для мережових запитів, які тривають певний час, щоб інтерфейс залишався відзивчивим.

- б) Яке призначення ключових слів `async` та `await`?

«`async`» позначає функцію, що повертає «`Promise`»; «`await`» призупиняє виконання функції до завершення «`Promise`» і повертає його результат у зручному лінійному стилі.

- в) Як виконати HTTP-запит за допомогою `fetch`? Яку структуру має відповідь?

Запит виконується через «`fetch(url)`», який повертає «`Promise`» з об'єктом «`Response`». Для JSON використовується «`response.json()`». Успішність перевіряється через «`response.ok`».

г) Навіщо використовується хук `onMounted`? На якому етапі він виконується?

«`onMounted`» викликається після монтування компонента в DOM, коли можна безпечно ініціювати завантаження початкових даних і взаємодіяти з елементами інтерфейсу.

д) Що таке стан завантаження (loading state) і як його реалізувати у Vue?

Це реактивний прапорець (наприклад, «`isLoading`»), який встановлюється в «`true`» перед запитом і скидається в «`false`» після завершення (у «`finally`»), щоб показати індикатор завантаження.

е) Як обробити помилку мережевого запиту за допомогою `try/catch`?

Код запиту розміщується в «`try`», а в «`catch`» зберігається повідомлення про помилку для відображення користувачу; блок «`finally`» використовується для скидання стану завантаження.

ж) Чим `computed` відрізняється від звичайної функції у Vue?

«`computed`» кешує результат і автоматично перераховується лише при зміні залежних реактивних даних, тоді як звичайна функція викликається щоразу без кешування.

и) Як реалізувати локальний список обраних без збереження на сервері?

На клієнті зберігається масив ідентифікаторів обраних елементів; додавання/видалення виконується перевіркою наявності «`id`» у цьому масиві.